



Lehrveranstaltung Weitverkehrsnetze

Kapitel 1 IP-Routing

Prof. Dr.-Ing. Jürgen Jasperneite

Fokus innerhalb des Netzes

Aufbau des Internets

Routing

- Internet setzt sich aus einer großen Anzahl autonomer Systeme zusammen.
 - **Distanzvektor**- und **Link-State**-Protokolle bilden die beiden IGP-Hauptklassen (IGP = Interior Gateway Routing Protocol).
 - Die Aufgabe dieser beiden Protokolltypen besteht darin, Routen durch autonome Systeme zu finden.
- **Distanzvektor**
 - Jeder Router konstruiert ein Feld (Vektor) mit den Distanzen (Kosten) zu allen Nachbarn
 - Vektor wird zu unmittelbaren Nachbarn distributiert.

Distanzvektorprotokolle – Ablauf

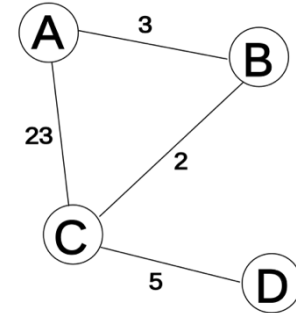
1. Erzeuge eine **Kostenmatrix**, welche Router über welche Nachbarn und zu welchen Kosten erreichbar sind. - Diese Matrix enthält anfangs nur die (bekannten) Kosten zu direkten Nachbarn.
2. Erzeuge eine **Aufstellung** mit Informationen, welche Router zu welchen Kosten am besten erreicht werden können und schicke sie an **alle Nachbarn**.
3. Warte auf Aufstellungen dieser Art von anderen Routern, rechne diese dann in die eigene Kostenmatrix ein.
4. Ändern sich dadurch die minimalen Kosten, zu denen ein Router erreicht werden kann: fahre mit Schritt 2 fort, sonst mit Schritt 3.

Suche nach kürzestem Weg: Bellman-Ford-Algorithmus

Richard Bellman, Lester R. Ford, beides Mathematiker (USA)

Beispiel

	von	via	via	via	via		von	via	via	via	via		von	via	via	via	via		von	via	via	via	via
	A	A	B	C	D		B	A	B	C	D		C	A	B	C	D		D	A	B	C	D
	zu A						zu A						zu A						zu A				
	zu B						zu B						zu B						zu B				
	zu C						zu C						zu C						zu C				
	zu D						zu D						zu D						zu D				
	von	via	via	via	via		von	via	via	via	via		von	via	via	via	via		von	via	via	via	via
	A	A	B	C	D		B	A	B	C	D		C	A	B	C	D		D	A	B	C	D
	zu A						zu A						zu A						zu A				
	zu B						zu B						zu B						zu B				
	zu C						zu C						zu C						zu C				
	zu D						zu D						zu D						zu D				
	von	via	via	via	via		von	via	via	via	via		von	via	via	via	via		von	via	via	via	via
	A	A	B	C	D		B	A	B	C	D		C	A	B	C	D		D	A	B	C	D
	zu A						zu A						zu A						zu A				
	zu B						zu B						zu B						zu B				
	zu C						zu C						zu C						zu C				
	zu D						zu D						zu D						zu D				
	von	via	via	via	via		von	via	via	via	via		von	via	via	via	via		von	via	via	via	via
	A	A	B	C	D		B	A	B	C	D		C	A	B	C	D		D	A	B	C	D
	zu A						zu A						zu A						zu A				
	zu B						zu B						zu B						zu B				
	zu C						zu C						zu C						zu C				
	zu D						zu D						zu D						zu D				



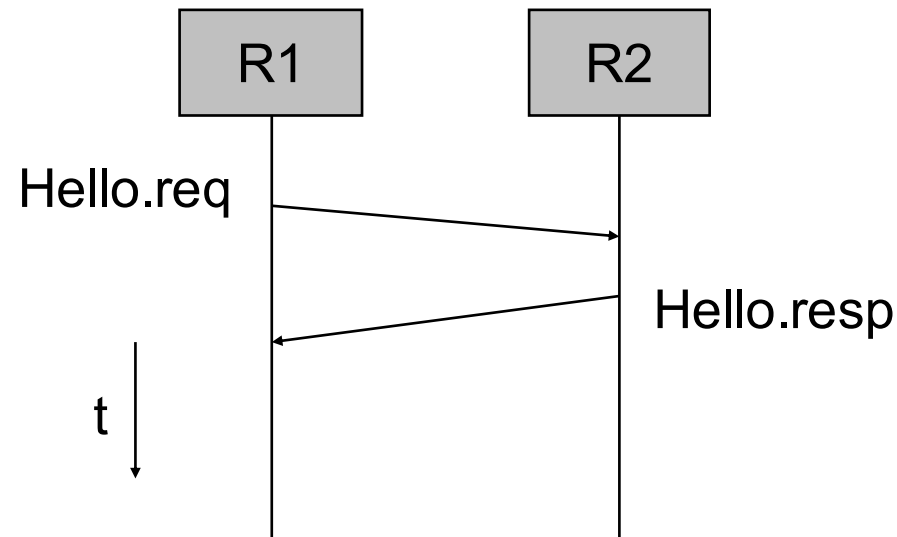
Link State Routing

Arbeitsschritte im Router bei Link-State Routing

1. Nachbarn entdecken und deren Netzadressen ermitteln
2. die Kosten zu jedem seiner Nachbarn messen
3. Router stellt ein Paket zusammen, in dem alles enthalten ist, was er in Schritt 1 und 2 gelernt hat
4. das Paket an **alle anderen Router** senden
5. den kürzesten Pfad zu allen anderen Routen berechnen.

Schritt 1: Ermittlung der Nachbarrouter

- Nach Start muss Router ermitteln, wer seine Nachbarn sind.
 - Hello Protokoll
 - Jeder Empfänger muss sofort mit einer eindeutigen Identität antworten.
 - Punkt-zu-Punkt / Multiaccess unterschiedlich



Schritt 2: Ermittlung der Leitungskosten

- Jeder Router muss Kosten zu seinem Nachbarn kennen oder schätzen
 - Echo-Nachrichten
 - Laufzeit: $t = \text{RTT} / 2$
 - Symmetrie wird vorausgesetzt

Schritt 3: Link-State-Pakete erstellen

- Zusammenstellen eines Pakets mit allen erfassten Informationen über
 - Nachbarn
 - Kosten
- Sequenznummer und Alter (wird später beschrieben)
- Wann sollen Link-State-Pakete erstellt werden?
 - Periodisch oder ereignisorientiert

Beispiel

.

Schritt 4: Link-State-Pakete verteilen

- Für richtige Funktionsweise ist **zuverlässige** Zustellung notwendig.
 - Router können ihre Wege nach Empfang der LS-Pakete ändern
 - Inkonsistenzen, Schleifen
- Basiskonzept zur Verteilung: **Flooding** (Fluten)
 - Jedes Paket enthält Folgenummer (Seq), mit jedem neuen LS-Paket:
 $Seq = Seq + 1$
 - Bei Empfang wird mit bekannter Liste verglichen
 - Neu: An alle Interfaces weiterleiten, ausser an dem empfangenen.
 - Alt oder Duplikat: verwerfen

Reliable Flooding

Schritt 4: Link-State-Pakete verteilen (2)

- Problemstellungen
 - Wiederverwendung von Seq-Nummern
 - 32 Bit : 137 Jahre bei 1 LS-Paket / sec.
 - Bei Routerausfall vergisst der Router seine Paketliste
 - Veränderung durch Bitfehler
- Ein Lösungsansatz*: Einführen eines Alters, welches jede Sekunde um 1 dekrementiert wird.
 - Alter = 0 : Router verwirft LS-Paket
 - Alter > 0: Verarbeiten und Weiterleiten

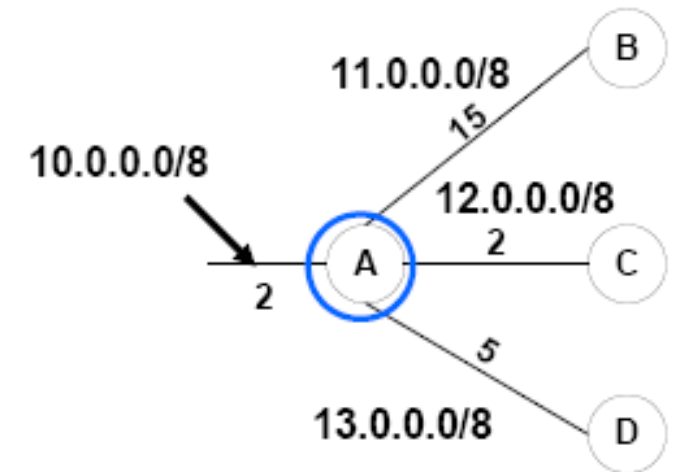
*Man kann natürlich auch mit 0 starten und inkrementieren und ein MaxAge definieren.

Schritt 5: Berechnung neuer Wege

- LS-Pakete werden in lokaler Topologiedatenbank des Routers gespeichert.
- Ausführung eines Algorithmus um die kürzesten Verbindungen zu allen möglichen Zielen ermitteln

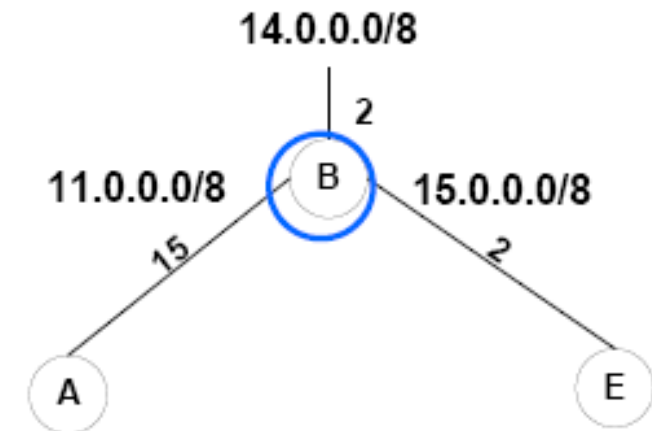
Beispiel: Schritte 1 und 2

- Router A tauscht *Hello*-Pakete mit:
 - Router B auf Netz 11.0.0.0/8 mit Kosten 15,
 - Router C auf Netz 12.0.0.0/8 mit Kosten 2,
 - Router D mit Netz 13.0.0.0/8 mit Kosten 5.
 - 10.0.0.0/8 ist ein Blatt-Netz mit Kosten 2
- Das ist die Link State Information, die Router A an alle anderen weitergibt.

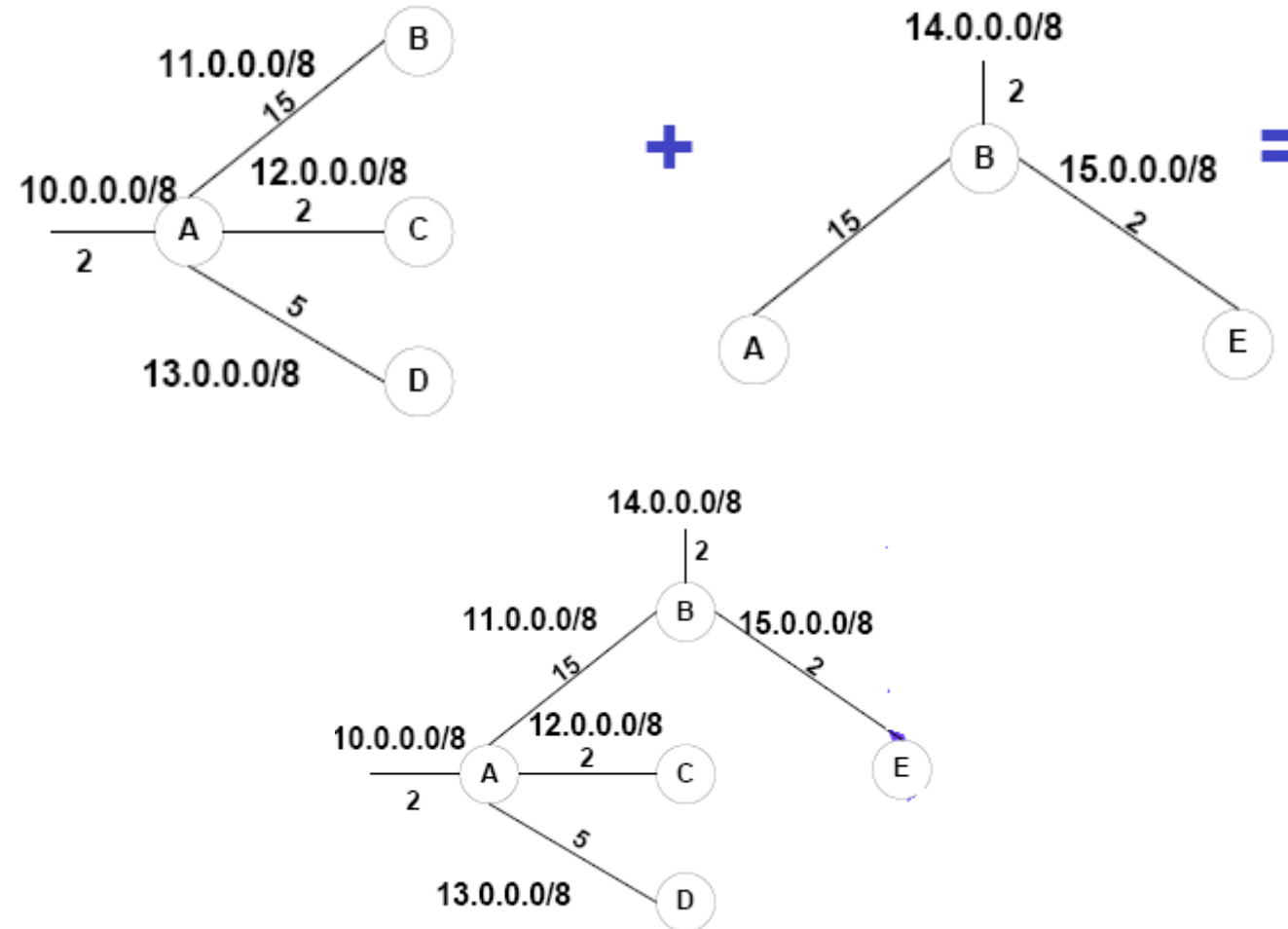


Link State Informationen von Router B

- **Router A erhält folgende Link-State Information von Router B:**
- Router A über Netz 11.0.0.0/8 zu Kosten von 15 verbunden
- Mit Router E über Netz 15.0.0.0/8 zu Kosten von 2 verbunden
- Blattnetz 14.0.0.0/8 mit Kosten von 2

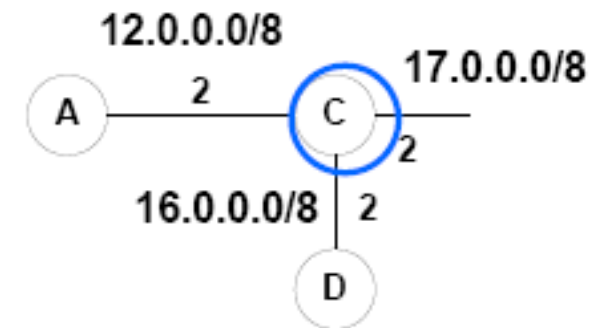


Berücksichtigung der Information von Router B

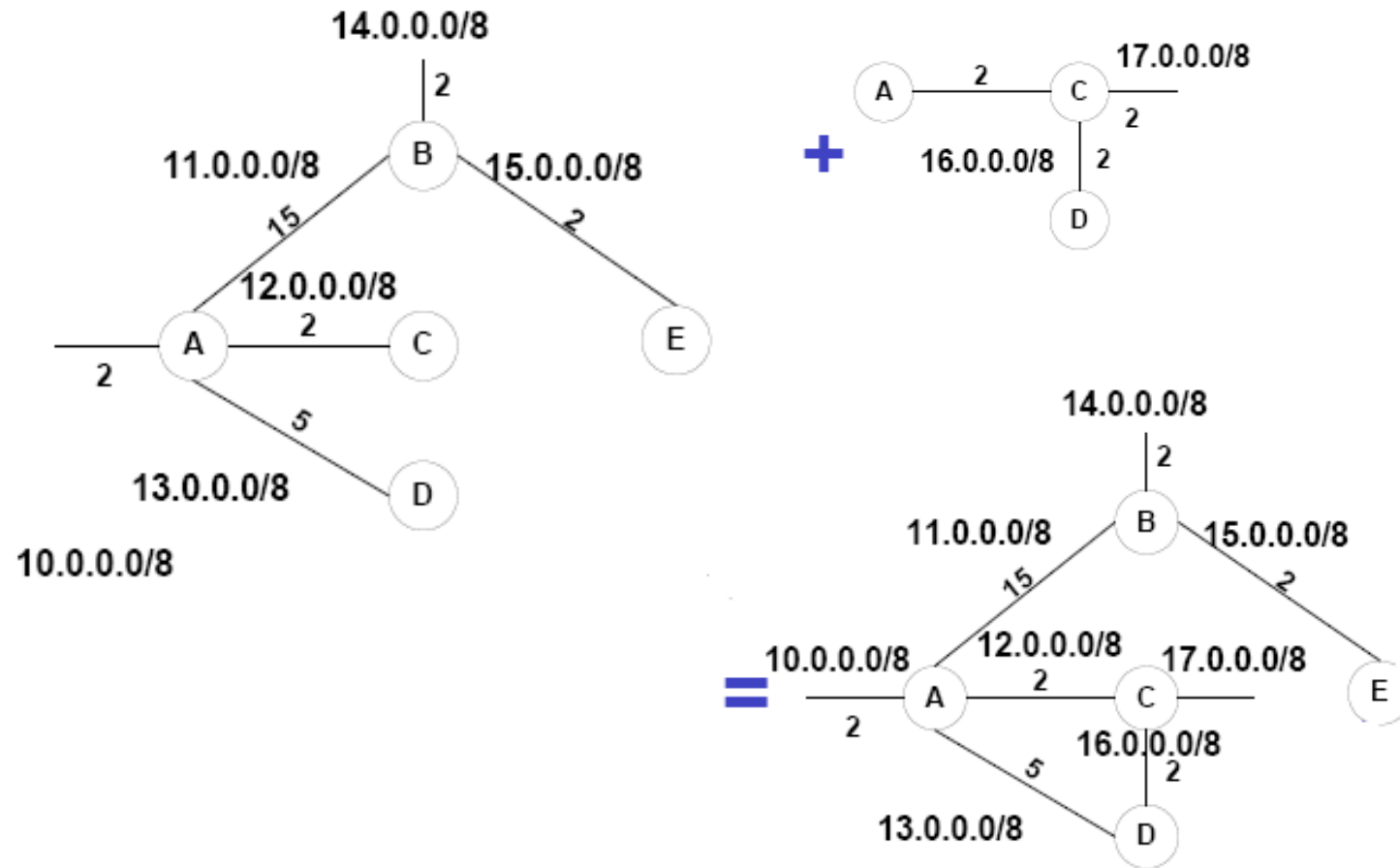


Link-State Information von Router C

- **Router A erhält folgende Link-State Information von Router C:**
- Mit Router A über Netz 12.0.0.0/8 zu Kosten von 2 verbunden
- Mit Router D über Netz 16.0.0.0/8 zu Kosten von 2 verbunden
- Blattnetz 17.0.0.0/8 mit Kosten von 2

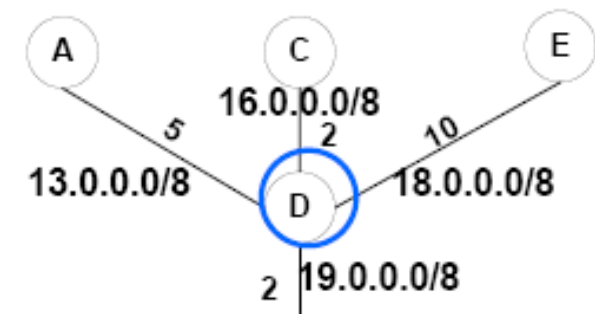


Berücksichtigung der Informationen von Router C

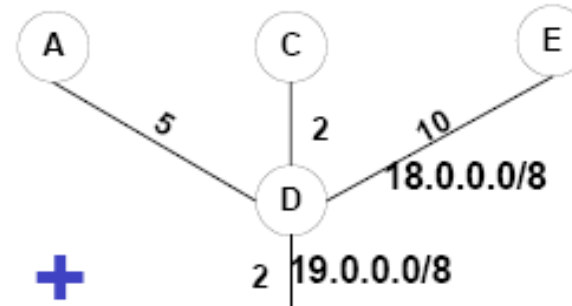
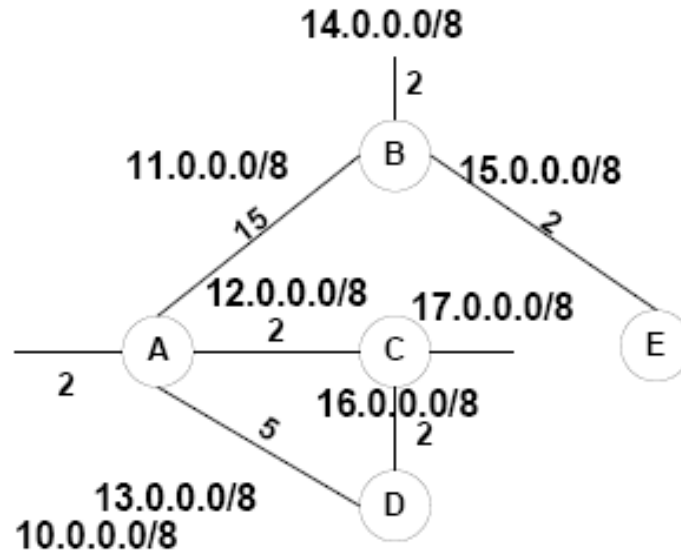


Link State Information von Router D:

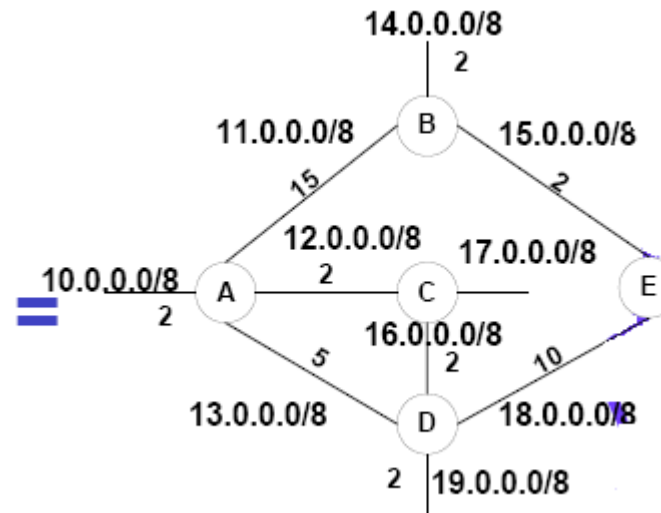
- **Router A erhält folgende Link-State Information von Router D:**
- Mit Router A über Netz 13.0.0.0/8 zu Kosten von 5 verbunden
- Mit Router C über Netz 16.0.0.0/8 zu Kosten von 2 verbunden
- Mit Router E über Netz 18.0.0.0/8 zu Kosten von 10 verbunden
- Blattnetz 19.0.0.0/8 mit Kosten von 2



Berücksichtigung der Information von Router D



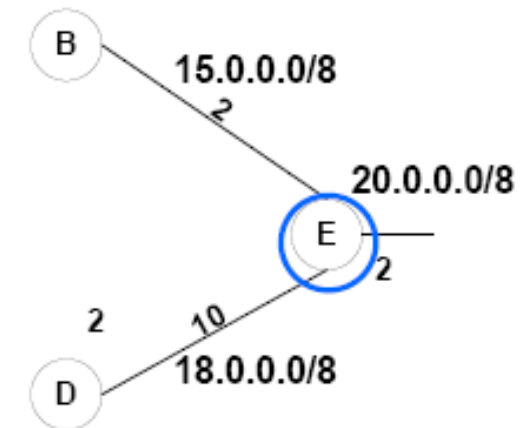
+



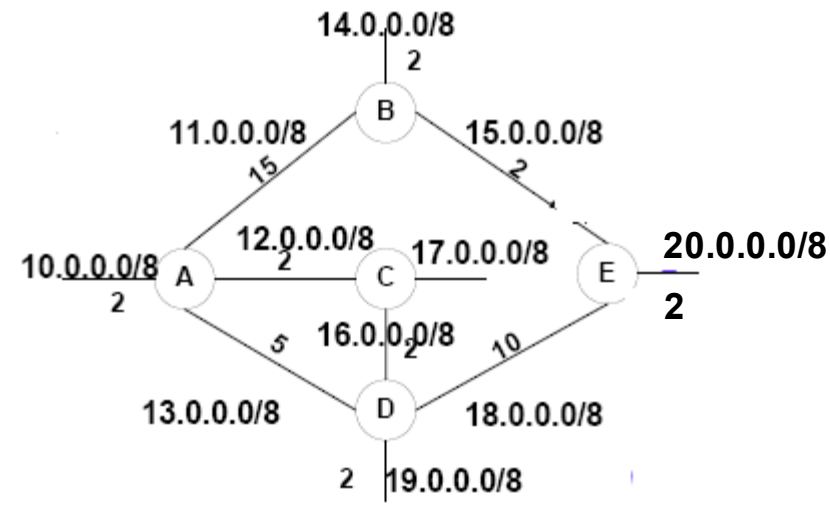
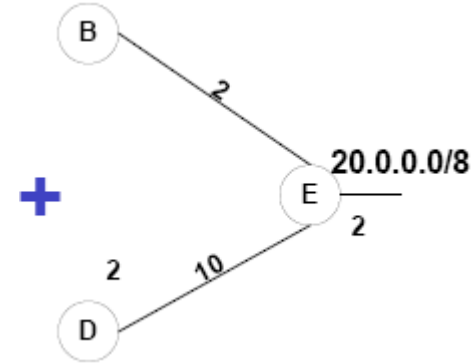
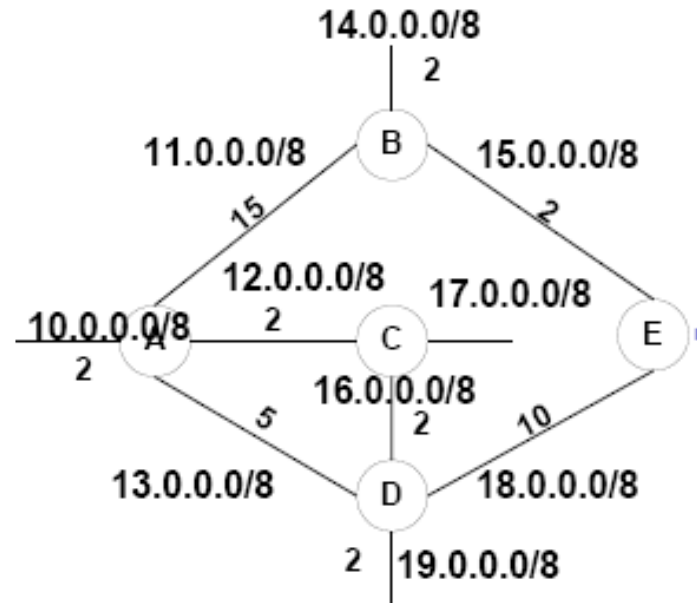
=

Link State Information von Router E

- **Router A erhält folgende Link-State Information von Router E:**
- Mit Router B über Netz 15.0.0.0/8 zu Kosten von 2 verbunden
- Mit Router D über Netz 18.0.0.0/8 zu Kosten von 10 verbunden
- Blattnetz 20.0.0.0/8 mit Kosten von 2

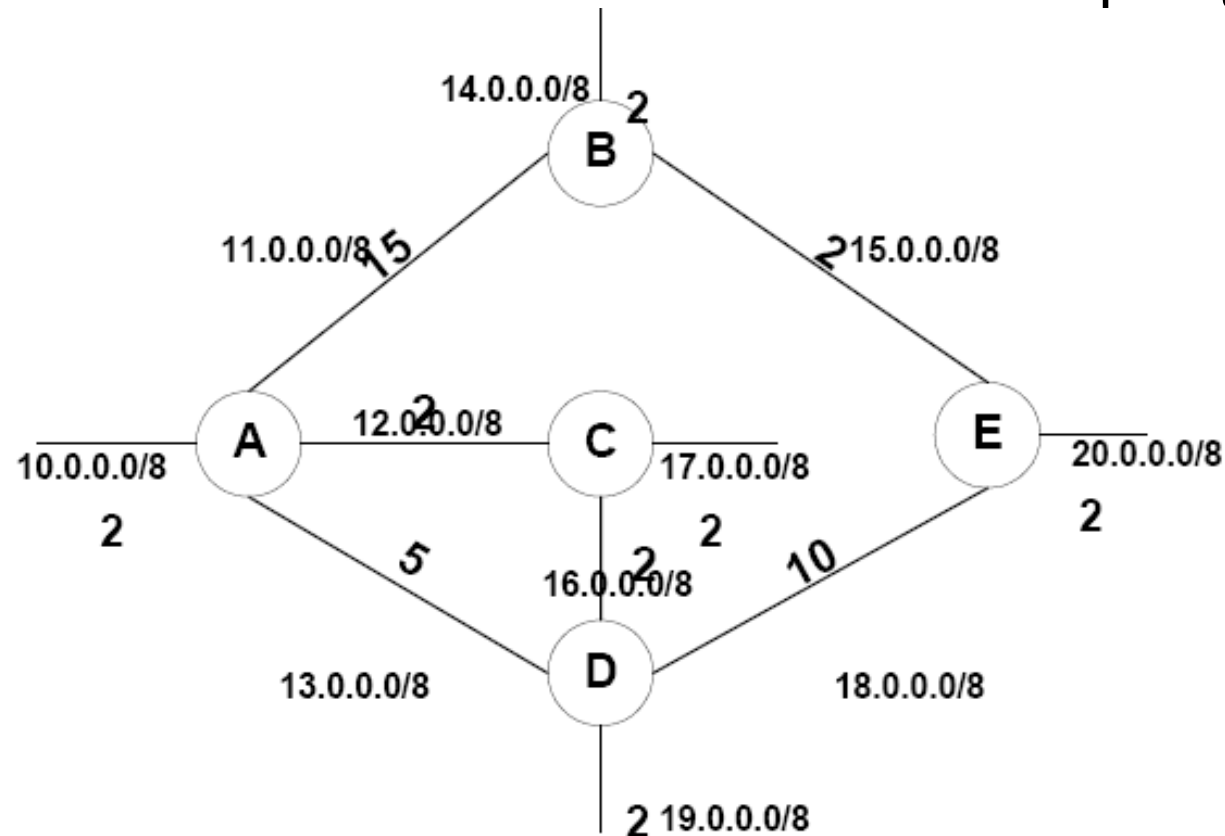


Berücksichtigung der Information von Router E



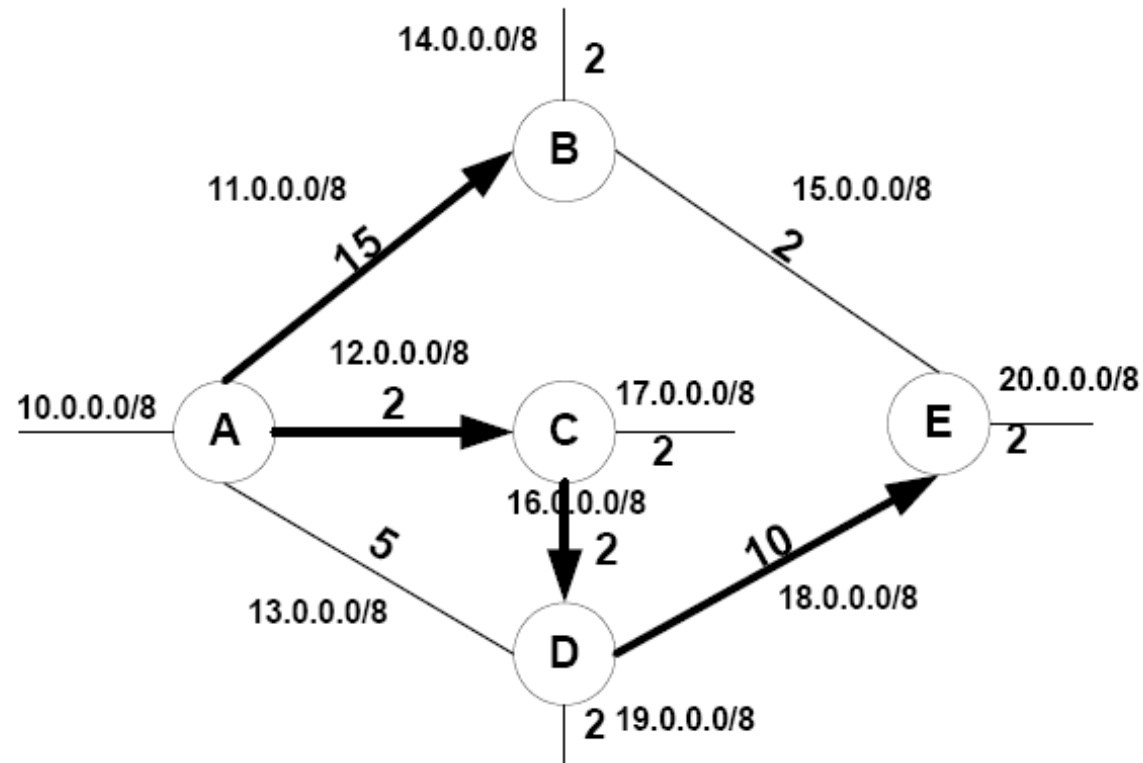
Schritt 5: Topologie

- Router A erstellt mit diesen Informationen eine Topologie des Netzes



Schritt 5: Wahl des besten Weges

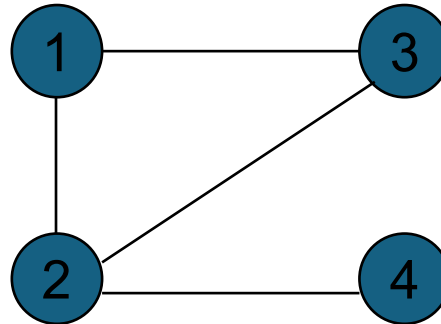
- Router A berechnet den besten Weg auf der Grundlage des SPT (Shortest Path Tree).



Baumalgorithmen zum Bestimmen kürzester Wege

Baumalgorithmen

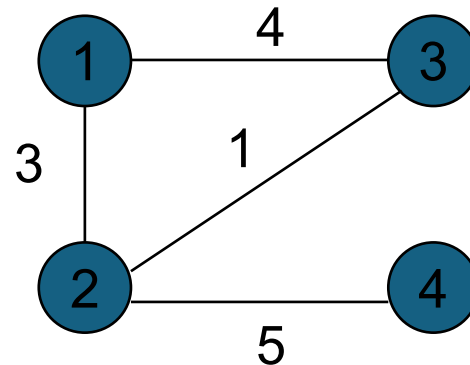
- Def.: Ein **Graph** G besteht aus einer nichtleeren **Knotenmenge** V und einer **Kantenmenge** E , sowie einer Abbildung ω (Inzidenzabbildung), die jedem Element aus E genau ein Knotenpaar i und j aus V zuordnet.



Domschke, Drexl: *Einführung in Operations Research*, Springer Verlag, 2005

Definitionen

- Ein Knoten j heisst **Nachfolger** eines Knotens i , wenn $[i, j]$ eine Kante in G ist. Die Menge der Nachbarn eines Knotens i bezeichnet man mit $N(i)$



- Ein Graph, dessen sämtliche Kanten eine Bewertung $c[i,j]$ besitzen, bezeichnet man als **kantenbewerteten Graph**. c seien hier die **Kosten** für den Transport von i nach j . Bewertete Graphen notieren wir mit $G=[V, E, c]$

Definitionen

- Sei $G = [V, E, c]$ ein kantenbewerteter Graph und $w = (j_o, \dots, j_t)$ ein Weg von G .
Die Summe aller Kantenbewertungen

$$c(w) := \sum_{h=1}^{t-1} c_{h,h+1}$$

bezeichnet man dann als **Länge des Weges** w .

- Einen Weg w_{ij}^* von G bezeichnet man als kürzesten Weg von Knoten i nach Knoten j , falls in G kein anderer Weg w_{ij} von i nach j mit

$$c(w_{ij}) < c(w_{ij}^*)$$

existiert.

Beispiel

Der Dijkstra¹-Algorithmus

- Baumalgorithmus, um kürzeste Entfernungen von einem Startknoten **a** zu allen Knoten eines bewerteten Graphen $G=[V, E, c]$ zu ermitteln.

Besitzt Graph n Knoten, so sei

- $D[1..n]$ mit $D[i]$ die kürzeste Entfernung von a nach i und
- $R[1..n]$ mit $R[i]$ der Vorgänger von i in einem kürzesten Weg von a nach i
- Baumalgorithmen, weil in $R[1..n]$ ein Baum kürzester Wege gespeichert wird und auf Iterationsverfahren basiert.

1) Edsger W. Dijkstra: 1930-2002, niederländischer Informatiker



1. Initialisierung:

- Liste mit unbesuchten Knoten erstellen
- Kosten zum Erreichen des Startknotens 1 auf 0 setzen.
- Kosten aller Knoten in G (außer Startknoten 1) auf unendlich setzen

2. Knoten mit geringsten Kosten im Netzwerk auf „aktuell besucht“ setzen

3. Addiere die Kosten des aktuell besuchten Knotens zu den Kosten entlang der Kanten zum Erreichen der Nachbarn.

4. Kosten der Nachbarknoten durch die berechneten Kosten aus Schritt 3 ersetzen, falls diese geringer sind.

5. Wurden alle Knoten besucht? Falls Nein, bei Schritt 2 weiter machen, ansonsten Ende.

- Menge markierter Knoten sei MK
- Startbedingung: $MK := \{1\}; D[1] := 0; \forall i \neq 1 : D[i] := \infty$

While MK $\neq \emptyset$ *do*

{

- Wähle den Knoten h aus MK mit
 $D[h] = \min \{d[i] \mid i \in MK\}$

- *for* (all) $j \in N(h)$ *do*
 If $D[j] > D[h] + c_{hj}$
 begin

$D[j] = D[h] + c_{hj}; R[j] := h; MK := MK \cup \{j\}$

end

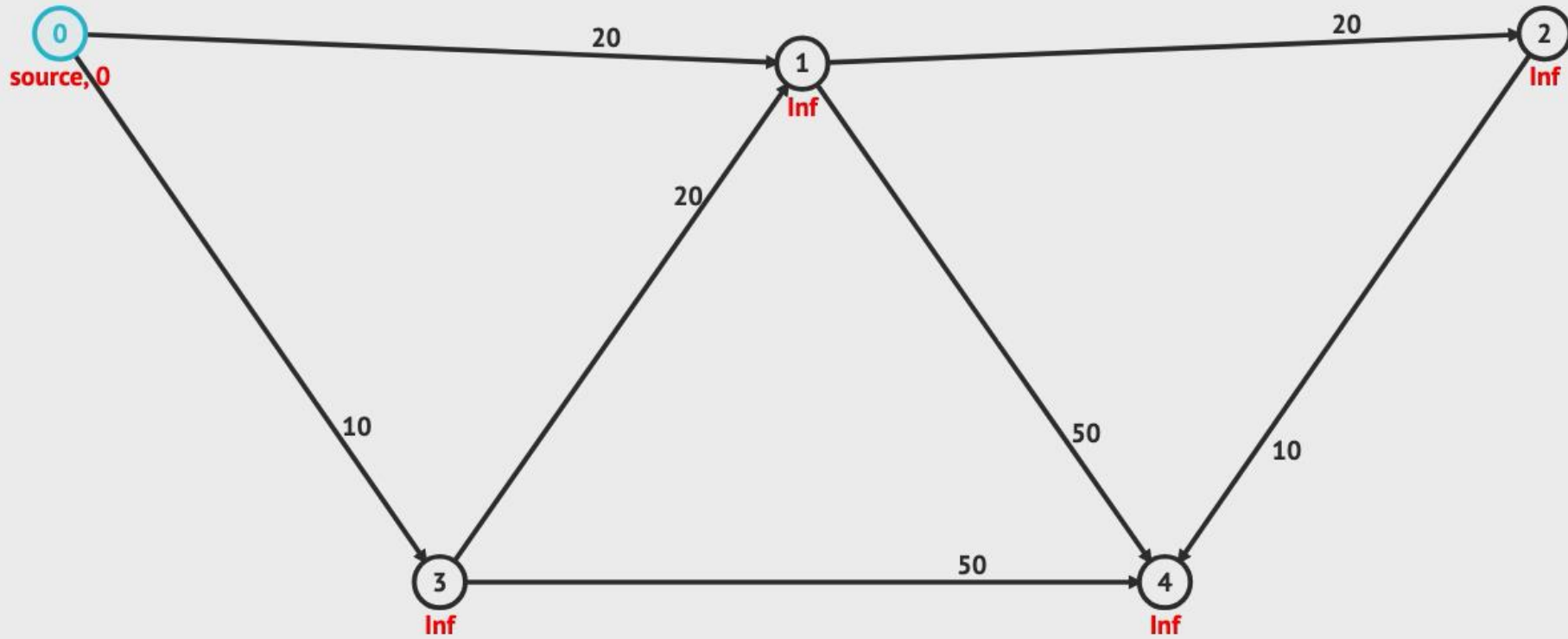
- Eliminiere h aus MK;

Abbruch, wenn MK = $\emptyset$

}

Ergebnis

- In $D[1..n]$ ist die kürzeste Entfernung von $a=1$ zu jedem anderen Knoten gespeichert.
- Aus $R[1..n]$ ist ein kürzester Weg von $a=1$ zu jedem Knoten *rekursiv* entwickelbar.



OriginalDijkstra(0)

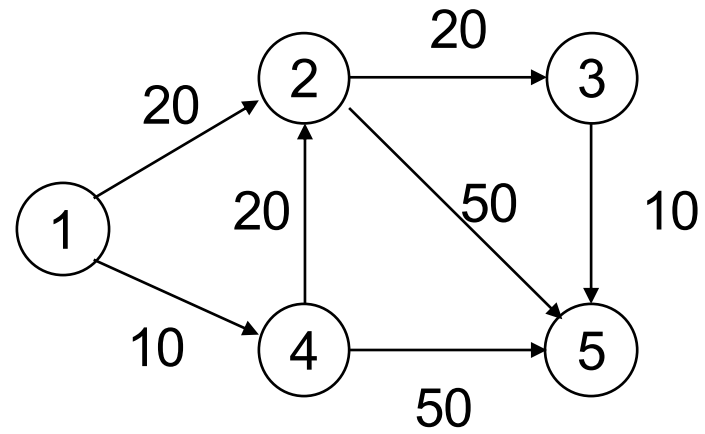
The current priority queue $\{(0,0), (999,1), (999,2), \dots\}$.
Exploring neighbors of vertex $u = 0$, $d[u] = 0$.

```

show warning if the graph has -ve weight edge
initSSSP, pre-populate PQ
while !PQ.empty() // PQ is a Priority Queue
    for each neighbor v of u = PQ.front(), PQ.pop()
        relax(u, v, w(u, v)) + update PQ

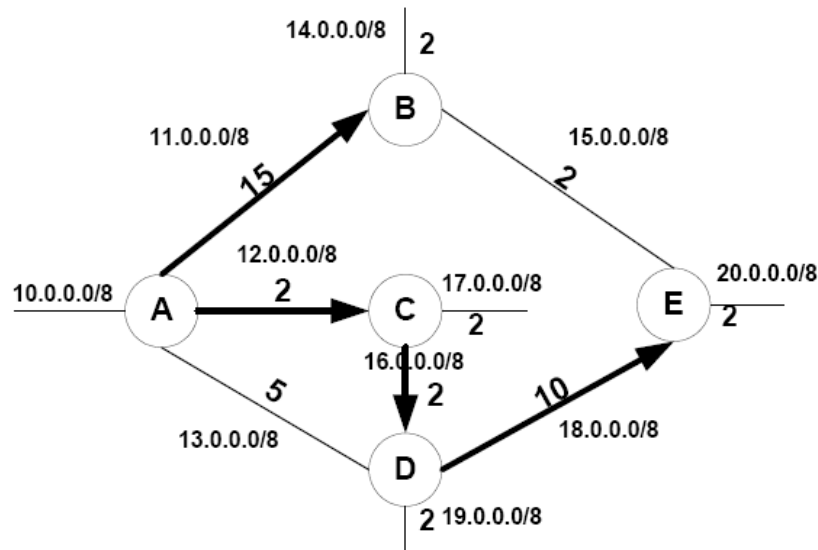
// dijkstra.cpp/py/java/ml
  
```

Beispiel



SPT Ergebnis aus Beispiel

- Das SPT-Ergebnis wird in die Routingtabelle über



Routingtabelle Router A

10.0.0.0/8	connected	e0
11.0.0.0/8	connected	s0
12.0.0.0/8	connected	s1
13.0.0.0/8	connected	s2

14.0.0.0/8	17	s0
15.0.0.0/8	17	s1
16.0.0.0/8	4	s1
17.0.0.0/8	4	s1
18.0.0.0/8	14	s1
19.0.0.0/8	6	s1
20.0.0.0/8	16	s1

Fragen

